

Merchant Web Architecture & Store Operations Blueprint

Detailed UI/UX components, order handling, store scope, catalog ownership, settlements, security, data contracts and monorepo implementation flow.

MERCHANT WEB

ADMIN WEB

CUSTOMER

DRIVER

FIREBASE

Architecture principles

- 1

Merchant scope is store-bound

Every read and write is limited to assigned approved store IDs.
- 2

Paid orders drive fulfillment

Merchant fulfillment begins only after trusted payment confirmation.
- 3

Order transitions are commands

Confirm, prepare, ready and rejection are server-validated.
- 4

Catalog edits preserve history

Mutable menus never rewrite item snapshots in historical orders.
- 5

Admin remains the control plane

Approvals, refunds, role changes and exceptional overrides remain audited.

DOCUMENT SCOPE

- 1

Role experience

Screens, commands, states and UX
- 2

Workflow ownership

Who reads, writes, approves and resolves
- 3

Data contracts

Canonical records, snapshots and enums
- 4

Security boundaries

Rules, Functions, App Check and audit
- 5

Monorepo flow

Shared packages without forced UI reuse
- 6

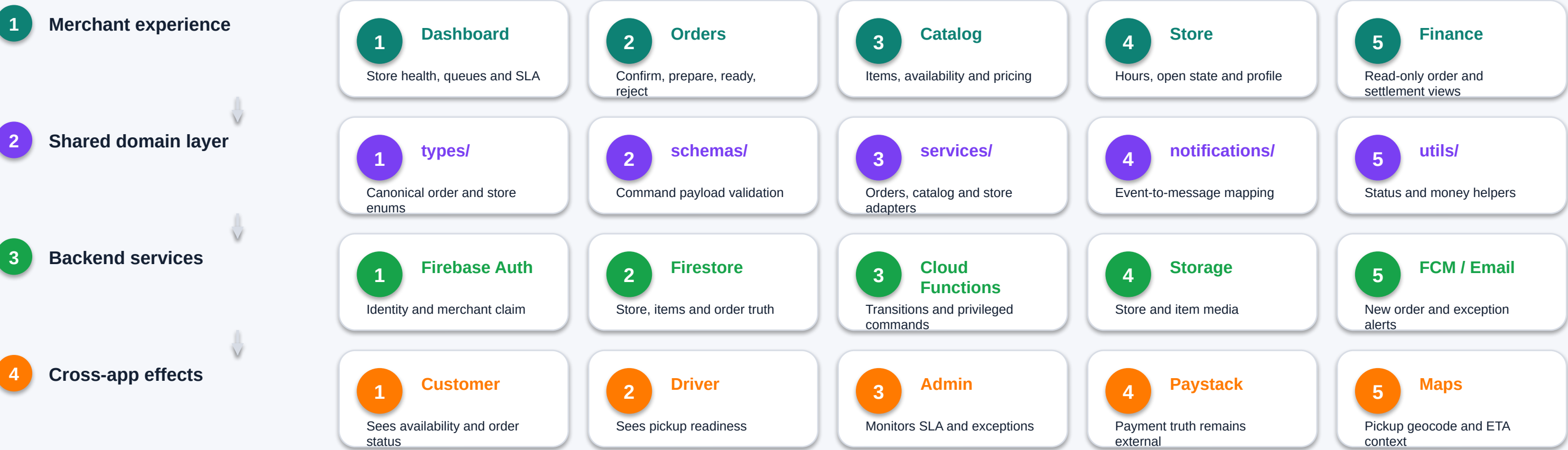
Development plan

Phases, tests, deployment and Codex prompts

VERSION 1.0 - JULY 2026

Merchant Web Role in the Five-App System

The merchant experience operates one or more approved stores without becoming a second source of platform truth.



Boundary rule

Merchant Web may manage only approved store scope. It cannot create arbitrary roles, mark a payment paid, assign drivers, process refunds or bypass the canonical order transition matrix.

Merchant Web Shell and Component System

Desktop-first store operations UI with fast queue handling, visible SLA context and safe action confirmation.

1 Page anatomy and navigation

1 Primary navigation

- Dashboard
- Orders
- Catalog
- Store profile
- Finance
- Support / Activity

2 Header controls

- Store selector when multi-store
- Open / closed indicator
- Notification center
- Account and logout

3 Operational surface

- KPI cards
- Queue tabs
- Search and filters
- Order rows / cards
- Context drawer

2 Reusable UX components

1 Status chips

Paid, New, Confirmed, Preparing, Ready, Rejected, Cancelled and issue labels combine text with semantic color.

2 Command controls

Confirm, Start preparing, Mark ready and Cannot fulfill are disabled while pending and require valid current state.

3 Feedback states

Skeleton, empty queue, offline banner, retry, inline validation, toast and full error details for operational failures.

4 Responsive behavior

On narrow screens, collapse navigation, stack KPIs and render orders as cards. Avoid admin-style wide tables on phones.

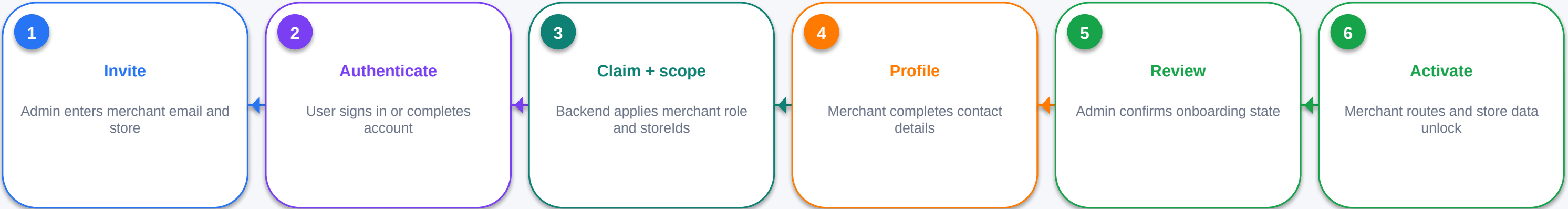
Accessibility baseline

Keyboard navigation, visible focus, 44 px targets, descriptive labels, non-color status text and confirmation for destructive or customer-impacting actions.

Merchant Identity, Store Assignment and Approval

Merchant access begins with an admin-controlled invitation and a precise store scope.

1 Onboarding command flow



2 State and capability matrix

State	Can sign in	Can read store	Can update orders	Can edit catalog	Admin action
invited	Limited	No	No	No	Resend / revoke
pending_profile	Yes	Read-only	No	No	Review profile
pending_approval	Yes	Read-only	No	Draft only	Approve / reject
active	Yes	Scoped	Allowed transitions	Scoped	Suspend / change scope
suspended	Blocked or read-only	No operational reads	No	No	Unsuspend
archived	No	Historical retained	No	No	Retention workflow

Claim consistency

Role claims and users/{uid}.storeScope must be updated through a privileged backend command and audited together. The UI must fail closed if they disagree.

Store Operations Dashboard

A focused overview that answers: what needs attention now, what is late and whether the store can accept orders.

1

Operational KPIs

1

New paid orders

Orders awaiting merchant confirmation; click filters the queue.

2

Preparing

Confirmed orders currently in production.

3

Ready for pickup

Orders ready and waiting for a driver.

4

SLA breaches

Confirmation or preparation timers beyond thresholds.

2

Store state

1

Open for orders

Explicit operational toggle separate from published active state.

2

Opening hours

Current schedule, next closing time and temporary closure reason.

3

Catalog health

Available items, unavailable items and missing-image or invalid-price warnings.

4

Integration status

Realtime connection, notification permission and last successful sync.

3

Attention feed

!

Merchant not responding

Escalation countdown and direct link to affected order.

!

Driver waiting

Ready order has an assigned driver at pickup location.

!

Customer or admin note

Unresolved issue requiring merchant response.

!

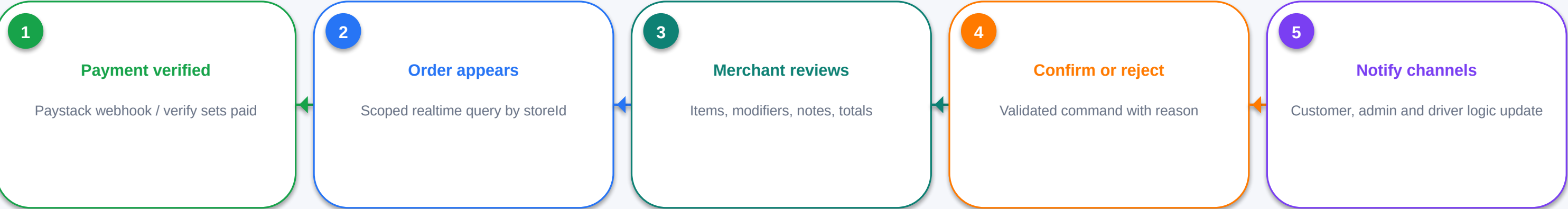
Store or item approval

Admin decision or requested correction.

Incoming Paid Order Queue and Confirmation

The merchant receives only orders whose payment state permits fulfillment.

1 Order intake flow



2 Queue contract

Field / control	Purpose	Source of truth	Failure handling
Order ID + timer	Identity and confirmation SLA	order + status timestamp	Highlight and create activity
Customer snapshot	Name / contact policy	order snapshot	Do not query mutable profile for history
Item snapshots	Production instructions	order.items[]	Cannot be edited after payment
Payment chip	Paid / failed / refund state	provider-confirmed order.payment	Merchant cannot change
Confirm	Move paid -> confirmed	Cloud Function command	Disable while pending; refresh canonical order
Cannot fulfill	Reject with code / note	Cloud Function command	Creates cancellation/refund workflow

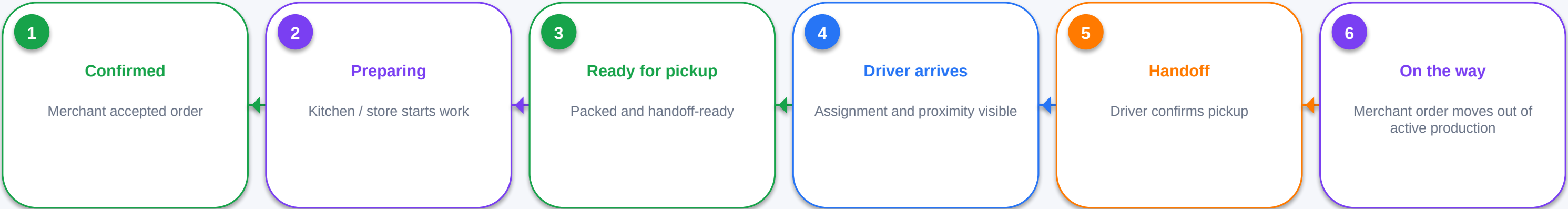
Critical invariant

A merchant UI callback, local cache or user claim cannot establish payment truth. Only verified provider data can allow fulfillment to begin.

Preparing, Ready and Pickup Handoff

Preparation state changes are explicit, timestamped and visible to every app.

1 Fulfillment progression



2 State ownership and UI behavior

Current state	Merchant action	Required validation	Writes	Customer / driver effect
confirmed	Start preparing	paid, same store, not cancelled	status + preparingAt	Customer sees preparing
preparing	Mark ready	active order, items fulfilled	status + readyAt	Driver receives pickup-ready alert
ready	Wait / report issue	No merchant jump to delivered	issue/activity only	Driver performs pickup
driver_assigned	Continue preparation	Assignment does not change merchant state	read only assignment	Shows driver identity / ETA
on_the_way	Archive from active queue	Driver pickup command verified	none	Customer tracking continues
cancelled	Acknowledge / add note	No further fulfillment	activity note only	Refund path managed elsewhere

Timestamp rule

Waiting time and SLA are derived from server timestamps. Never use a browser timer as the authoritative value.

Rejection, Cannot-Fulfill and Issue Resolution

Merchant exceptions provide structured facts; admin and payment services own financial consequences.

1 Merchant-originated exceptions

1 OUT_OF_STOCK

One or more paid item snapshots cannot be fulfilled. Include affected item IDs and optional substitute note.

2 STORE_CLOSED

Store cannot fulfill despite published state. Close ordering and supply a reason.

3 CAPACITY_LIMIT

Unexpected operational overload; provide expected recovery time.

4 ORDER_DETAIL_PROBLEM

Ambiguous modifier, customer note or impossible request; route for support review.

2 Resolution ownership

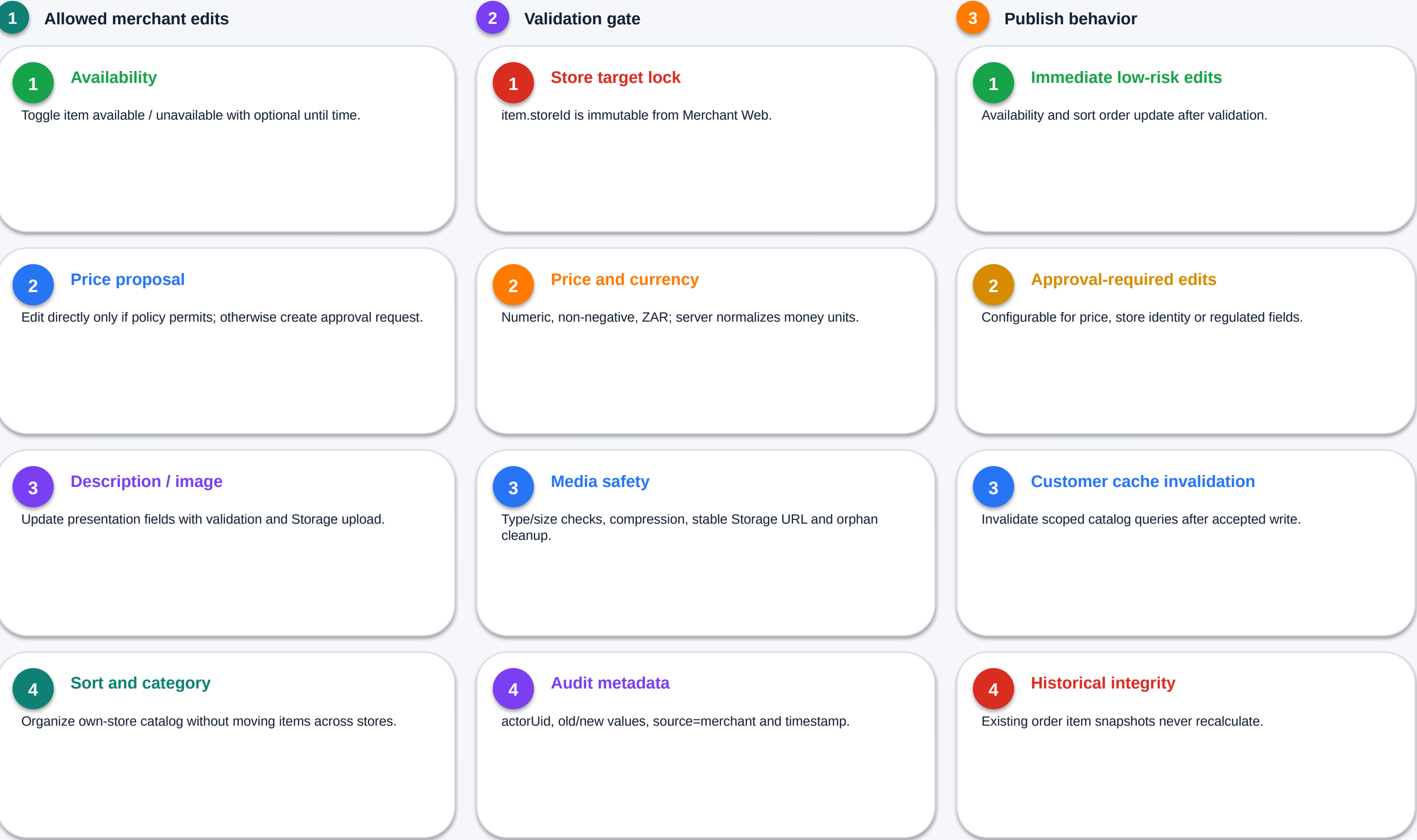
Issue	Merchant	Admin / system	Payment
Cannot fulfill	Reason + evidence	Cancel / resolve	Refund if captured
Partial item issue	Propose correction	Approve customer-safe resolution	Adjustment only through backend
Driver late	Keep order ready; report	Contact / reassign	No direct effect
Customer unreachable	Report attempt	Support intervention	No direct effect
Duplicate order suspicion	Pause and report	Investigate idempotency	Verify references
Integration failure	Retry / offline fallback	Activity + function logs	Reconcile event

Refund boundary

Merchant Web never calls Paystack refund APIs and never writes paymentStatus/refundStatus directly. It only submits a fulfillment exception command.

Scoped Catalog and Availability Management

Merchant edits accelerate daily operations while preserving admin governance and historical order snapshots.



Store Profile, Hours and Ordering Availability

Published identity, operational availability and temporary closure are separate states.

1 Store profile contract

1 Identity

Name, category, description, source and admin approval state.

2 Location

Address, placeId / Plus Code, GeoPoint and validated pickup entrance.

3 Operations

Opening schedule, special hours, openForOrders and prep-time estimate.

4 Commerce

Minimum order, fee-rule references and customer review visibility.

5 Media

Card and hero images with attribution metadata.

2 State model and effects

State	Customer visibility	Can receive orders	Who changes
pending	No / preview only	No	Admin approval
active + open	Yes	Yes	Admin activates; merchant opens
active + closed	Yes, closed	No	Schedule / merchant temporary toggle
inactive	No ordering	No	Admin
suspended	No operational access	No	Admin / Super Admin
archived	Historical reference only	No	Privileged command

Availability consistency

Checkout revalidates active, openForOrders, item availability, hours and fee rule. Customer catalog display alone is not an authorization to place the order.

Order Ledger, Commission and Settlement Views

Merchant finance screens explain platform deductions but never become the accounting source of truth.

1 Per-order financial snapshot

1 Customer subtotal

Sum of immutable item snapshots and modifiers.

2 Store commission

Versioned percentage / fixed rule captured at order time.

3 Delivery and service fees

Displayed separately; ownership follows platform policy.

4 Refund / adjustment

Provider-confirmed events and audited administrative reasons.

5 Net store amount

Derived from the captured financial snapshot, not current settings.

2 Settlement data contract

Field	Meaning	Write authority
orderId / paymentReference	Traceability to canonical order and provider transaction	System
grossAmount	Captured customer amount	Paystack verification
commissionSnapshot	Rule/version used for this order	Backend pricing service
refundAmount	Confirmed provider refund	Webhook / reconciliation
netMerchantAmount	Derived settlement basis	Backend ledger
settlementStatus	unsettled / scheduled / paid / disputed	Finance backend / admin
settledAt / reference	Payout proof when implemented	Backend

V1 decision

If automated merchant payouts are not implemented, present a read-only ledger and manual reconciliation status. Do not imply that a payout occurred without a trusted payout reference.

Operational Alerts, Timers and Offline Behavior

Alerts are event-driven and deduplicated; dashboard timers are derived from server time.

1 Merchant notifications

1 New paid order
High-priority push/browser notification with order link.

2 Confirmation SLA warning
Escalates before MERCHANT_NOT_RESPONDING threshold.

3 Driver arrival / waiting
Ready order and assigned driver proximity context.

4 Admin or support update
Resolution, suspension or requested store correction.

2 Delivery guarantees

1 Event idempotency
One logical event creates one notification per recipient/channel.

2 Read state
notifications/{id} tracks recipient, event, readAt and deep link.

3 Escalation activity
Final delivery failure becomes an admin activity, not a silent drop.

4 Quiet duplication control
Web and push channels share an event key.

3 Offline UX

1 Connection banner
Clearly show disconnected / reconnecting state.

2 No false success
Commands remain pending until canonical response arrives.

3 Refresh after resume
Requery active orders and invalidate stale cache.

4 Optional printer / sound
Treat as secondary alert; never depend on it for data state.

Merchant Read/Write Contract

Merchant Web reads a scoped operational projection and submits commands; it does not directly own canonical truth.

1

Readable data

Path / projection	Merchant scope	Purpose
users/{uid}	self	profile, role and storeScope
stores/{storeId}	assigned storeIds	identity, hours, operational state
items/{itemId}	where storeId in scope	catalog and availability
orders/{orderId}	storeId in scope	fulfillment queue and history
notifications/{id}	recipientUid=self	alerts and read state
activities/{id}	related store + permitted types	support / issue status
settlements or ledger projection	assigned storeIds	read-only financial view

2

Command writes

1

merchantConfirmOrder

Validates actor, store scope, payment state and current status; writes confirmedAt and history.

2

merchantStartPreparing

Allowed only from confirmed; server timestamp and audit.

3

merchantMarkReady

Allowed only from preparing; emits driver/customer notifications.

4

merchantRejectOrder

Structured reason; creates cancellation/refund workflow.

5

merchantUpdateCatalog / Store

Schema-validated, scoped and audited updates.

Snapshot principle

Orders store customer, store and item display snapshots. Merchant profile or catalog edits affect future reads and orders, not the meaning of an existing transaction.

Merchant Security and Privileged Boundaries

Route guards improve UX; Firebase rules and server commands enforce store ownership.

1 Client defenses

1 Auth route guard

Require authenticated merchant role and active status.

2 Capability guard

Hide actions outside current state or policy.

3 Schema validation

Validate forms and command payloads before sending.

4 App Check

Attach valid attestation where supported; do not treat it as user authorization.

2 Backend enforcement

1 Deny by default

Firestore rules verify role, active status and store membership.

2 Immutable fields

Merchant cannot alter payment, customer IDs, storeId or protected status fields.

3 Functions

Order transitions, role changes, refunds and destructive actions run server-side.

4 Audit logs

Actor, resource, before/after, command ID and result are immutable.

3 Threats to test

1 Cross-store query

Merchant attempts another storeId.

2 Status jump

Direct ready/delivered write or stale command replay.

3 Payment tampering

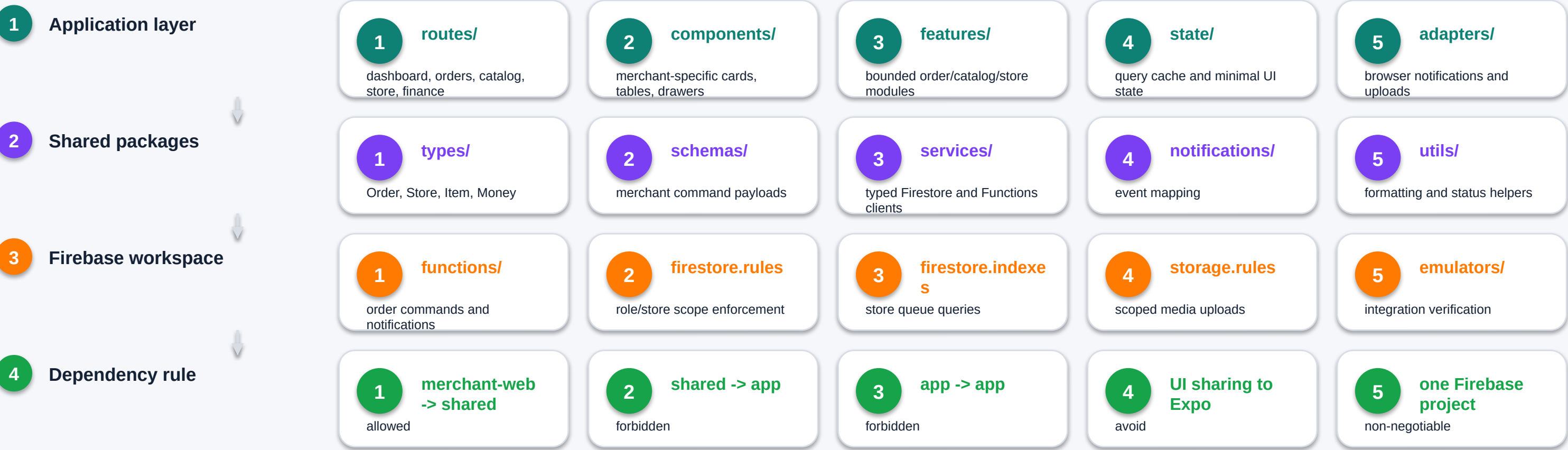
Client tries to mark paid or alter total.

4 Suspended session

Existing token attempts continued reads/writes.

Merchant Web Monorepo Boundaries

Merchant Web uses web-native UI while importing shared domain contracts and Firebase adapters.



Suggested route structure

apps/merchant-web/src/{routes,features,components,providers}; packages/shared/{firebase,types,schemas,services,notifications,utils}. Confirm actual repo paths before moving files.

Performance, Reliability and Test Matrix

Operational correctness is prioritized over decorative complexity.

1 Performance

1 Scoped listeners

One active order query per visible store/view; unsubscribe on route change.

2 Pagination

Server-backed history and catalog lists; bounded date ranges.

3 Media

Compressed images, thumbnails and lazy loading.

4 Code splitting

Lazy-load finance, settings and heavy editors.

2 Reliability

1 Idempotent commands

Stable command IDs protect confirm/reject/ready retries.

2 Transactions

Couple status, history, activity and notification outbox safely.

3 Canonical refresh

Replace optimistic operational state with returned order.

4 Failure visibility

Provider/function failures create an activity and actionable error.

3 Verification

1 Unit

Schemas, transition matrix, money and SLA helpers.

2 Emulator

Firestore rules, Functions and multi-role scope tests.

3 Web E2E

Invite, order confirmation, ready, reject, catalog and offline resume.

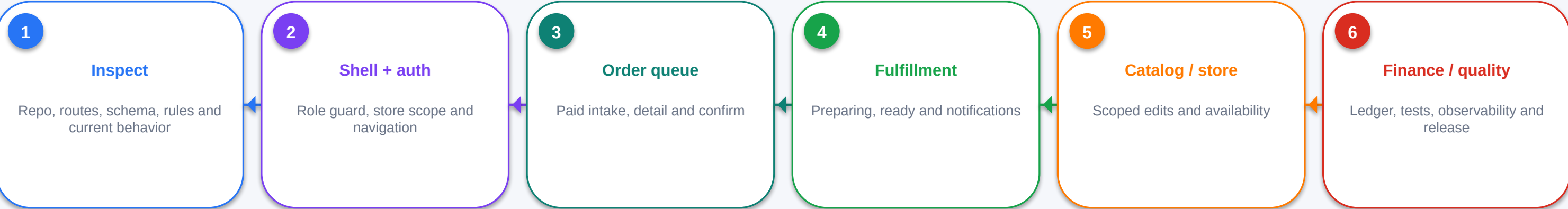
4 Release gate

Typecheck, lint, build, rules/index validation and manual smoke.

Merchant Web Development Roadmap

Implement in bounded phases that preserve the shared backend and current working apps.

1 Phased build sequence



2 Phase exit criteria

Phase	Exit criteria	High-risk checks
1. Discovery	Exact files, paths, indexes, Functions and data names documented	No assumptions overwrite deployed truth
2. Identity	Merchant can sign in only to assigned approved stores	Cross-store reads/writes denied
3. Orders	Paid orders display and confirm exactly once	No client payment truth; stale command rejected
4. Fulfillment	Preparing/ready visible to customer, driver and admin	Allowed transitions and server timestamps
5. Catalog/store	Scoped edits, validation, cache invalidation and audit	Historical snapshots unchanged
6. Finance/release	Ledger reconciles; tests/build/rules pass	No unverified payout or refund claims

Reusable Codex instruction

Resume current progress. Inspect merchant-web and shared backend first; list exact files/data paths; implement one bounded feature; preserve one Firebase project; run targeted tests, typecheck/lint/build; report every change and deployment impact.

Merchant Web Traceability and Decisions

Traceability, current decisions and implementation assumptions.

1

Source-to-module traceability

Source	Captured architecture detail	Used in this document
Admin blueprint - system context	Five apps, shared packages, one Firebase backend	Pages 2, 13, 15
Admin blueprint - order lifecycle	Merchant owns confirm, prepare, ready and rejection	Pages 6-8
Admin blueprint - canonical order	Order/payment/delivery snapshots and provider enrichment	Pages 6-13
Admin blueprint - stores/items	Manual/admin workflows and scoped catalog governance	Pages 9-10
Admin blueprint - security	Roles, store scope, Functions, audit and denial by default	Pages 4, 13-16
Current project observations	Merchant role exists in the shared five-app workflow	All module assumptions
Official provider guidance	Backend payment initialization, verification and signed webhook handling	Payment boundaries

2

Normalized decisions

One backend

Merchant Web uses the existing Firebase project and collections.

Store scope

Merchant access is limited to assigned approved store IDs.

Payment truth

Paystack verification/webhooks, not UI callbacks, authorize fulfillment.

Refunds

Merchant raises exceptions; admin/backend controls refund commands.

UI stack

React web remains web-native; shared code is domain logic, not mobile UI.

V1 finance

Read-only ledger unless trusted payout automation exists.

Source precedence

Current repo/deployment > approved architecture > mockup intent.

Document status

Architecture synthesis for implementation. Before changing code, validate exact collection names, existing Cloud Functions, deployed rules, current routes and environment variables against the monorepo.